Iteration 004: complete process record

From simulated genotypes to the final analysis — every step, in order

fmbenchmark

August 23, 2026

Abstract

This document records exactly how the Iteration 004 fine-mapping benchmark results were produced. It is written to be followed by someone who has not seen the project before: every stage names the file that implements it, the parameters it was run with, what it consumed and what it produced. Where a step went wrong during execution and had to be redone, that is recorded too, in §13 — a process record that omits its failures cannot be audited. The companion document `iter004_full_analysis.pdf` presents the results; this one explains how they came to exist.

Contents

1 Overview

The pipeline has nine stages. Each is a separate script; each writes its output to disk before the next begins, so any stage can be re-run without repeating the ones before it.

	stage	implemented in	produces
1	design grid	`generate_params_grid.R`	`params_grid.csv`
2	genotypes	`simulate_genotypes.R`	region panel (in `sim.rds`)
3	annotations	`simulate_phenotypes.R`	annotation matrices
4	causal selection	`simulate_phenotypes.R`	`truth$causal_indices`
5	phenotypes	`simulate_phenotypes.R`	y, β, realised PVE
6	summary stats + LD	`simulate_phenotypes.R`	z, LD matrix
7	method fitting	`run_methods.R` + 18 wrappers	`results.rds`
8	collection	`iter004_collect.R`	L1 fit table
9	analysis	`iter004_report.R` + 3 sense scripts	L2, L3, sense outputs

Determinism. Stages 2–6 run under `seed = 1000 + row_index`, set once per design row. Given the same grid and the same VCF inputs, the simulated data are reproducible exactly. Stage 9's cross-fitted test uses `set.seed(20260806)`. Stage 7 is not seeded centrally: each method controls its own randomness, and the BEATRICE family's Python trainer sets `torch.manual_seed(1)` internally.

2 Software environment

R	4.5.2 (`R/4.5.2-gfbf-2025b`)
Python	3.12.3 (`Python/3.12.3-GCCcore-13.3.0`)
GSL	2.8 (`GSL/2.8-GCC-14.3.0`)
scheduler	PBS Pro, queue `v1_small72a`, 72 h walltime
toolchain root	`/rds/general/project/neurogenomics-lab/live/tools`

The toolchain root holds a Python virtualenv (`fmpy-venv`), a wrapper (`py-venv-runner.sh`) that every Python-backed method is handed in place of a bare interpreter, the compiled binaries `finemap_v1.4.2` and `PAINTOR`, and the source checkouts `SparsePro`, `fine-mapping-inf` and `Funmap`. The last of these must be *built*, not merely cloned: its `finemapinf` sub-package declares a C extension.

This location matters. It must be on permanent storage. An earlier execution placed it behind a symlink onto a filesystem subject to a rolling 30-day purge; see §13.

Rebuild it with:

```
DEST=/rds/general/project/neurogenomics-lab/live/tools \
  bash scripts/hpc/rebuild_toolchain.sh
bash scripts/hpc/check_toolchain.sh      # must print RESULT: PASS
```

`check_toolchain.sh` derives the required Python imports by walking the tool sources with `ast`, subtracting the standard library, and importing what remains. It does not use a hand-written package list, because a hand-written list is what caused the failure in §13.

3 Stage 1 — the design grid

`scripts/hpc/generate_params_grid.R` writes 45 rows to `params_grid.csv`. Each row is one combination of architecture, annotation type, enrichment fold and (for the infinitesimal arm) p_{causal}. The within-row sweep is fixed:

S	$1, 2, 3, 5, 10$
ϕ	$0.05, 0.1, 0.2, 0.4, 0.6$
iterations	10
regions	10, nominal sizes $\{500, 500, 750, 750, 1000, 1000, 1500, 1500, 2000, 2000\}$
n	5000
LD	in-sample only (`n_ref = NA`)
annotations	10 per region

$45 \times 5 \times 5 \times 10 = 11{,}250$ scenarios; $\times 10$ regions $= 112{,}500$ fits per method.

The grid is regenerated at every submission, never reused from disk. It is untracked and persists between iterations, and an earlier run silently reused a stale 135-row grid from Iteration 003. The submitter now regenerates it unconditionally and echoes the resulting design before calling `qsub`, so a mismatch is caught in seconds rather than after 72 hours.

4 Stage 2 — genotypes

`R/simulate_genotypes.R` draws each region from real 1000 Genomes loci under `data/vcf`, filtered to MAF ≥ 0.01, then standardises columns.

Regions can be smaller than requested. If a locus does not contain enough variants passing the filter, the region is whatever is available. Measured over the 709 distinct regions actually used:

nominal	median actual	ratio	within 5%
500	499	1.00	100%
750	749	1.00	96%
1000	999	1.00	97%
1500	1483	0.99	62%
2000	1590	0.80	26%

The analysis uses the *nominal* size as its design factor. Variation in actual size within a nominal class is absorbed into the between-region variance component σ^2_u and correctly subtracted by the noise correction, so the design remains valid; only the label is approximate.

Regions are shared across the whole row. All 250 scenarios of a design row use the same ten regions, simulated once and cached in `sim.rds`. This is what creates the split-plot structure the analysis corrects for, and it is why two regions exist per size class — without a replicate draw, σ^2_u would not be identifiable.

5 Stage 3 — annotations

Ten annotations per region. Binary annotations are Bernoulli indicators; continuous ones are drawn from a shared-factor model so that annotations within an enrichment group are correlated. The same matrix is passed to every annotation-aware method, so no method sees information another does not.

6 Stage 4 — causal variant selection

`select_causal_variants()` in `R/simulate_phenotypes.R`.

- **No annotations**: S indices drawn uniformly without replacement.
- **With annotations**: weights $w_j = \exp\big(A_j^\top \log \gamma\big)$, normalised to probabilities, then S drawn without replacement.

γ is the enrichment vector for that row. `polyfun_oracle` reconstructs $\pi_j \propto \exp(A_j^\top \log \gamma)$ from the stored truth, using the identical formula — which is what makes it a true ceiling rather than a strong competitor.

7 Stage 5 — phenotypes

7.1 Sparse

Effects $\beta \sim N(0, \sigma_\beta^2)$ on the S causal variants; the residual variance is then chosen to hit the target heritability exactly:

$$\sigma_\varepsilon^2 = \mathrm{Var}(X\beta)\frac{1-\phi}{\phi} \quad \Longrightarrow \quad \mathrm{PVE} = \phi \text{ exactly.}$$

7.2 Sparse plus infinitesimal

A second component is drawn on the **non-causal variants only**, $\alpha_{nc} \sim N(0, 1/m_{nc})$ (the BEATRICE formulation; `inf_model = "beatrice"`, the default, is what was used). The components are then rescaled so that

$$\mathrm{Var}(g_{\text{sparse}}) = p_{\text{causal}}\phi, \quad \mathrm{Var}(g_{\text{inf}}) = (1-p_{\text{causal}})\phi, \quad \sigma_\varepsilon^2 = 1-\phi,$$

and β is rescaled by the same factor so the recorded truth stays consistent with the realised phenotype.

Consequence for interpretation. Every non-causal variant then has a genuine non-zero effect, while the ground truth used for scoring is the S sparse causals only. A method that detects infinitesimal signal is counted as making a false discovery. Absolute FDR is therefore not comparable between the two arms; *degradation* between arms is the meaningful quantity.

8 Stage 6 — summary statistics and LD

`compute_summary_statistics()`: marginal OLS per variant on the centred phenotype,

$$\hat\beta_j = \frac{x_j^\top y_c}{x_j^\top x_j}, \quad \mathrm{RSS}_j = \|y_c\|^2 - \hat\beta_j x_j^\top y_c, \quad \mathrm{se}_j = \sqrt{\frac{\mathrm{RSS}_j}{(n-2)\,x_j^\top x_j}}, \quad z_j = \hat\beta_j/\mathrm{se}_j.$$

LD is `cor(X)` on the *same* genotype matrix that generated the phenotype — in-sample throughout, with no reference panel anywhere in this iteration. Iteration 003 established that in-sample and reference-panel LD can invert method rankings, so no conclusion here transfers to the panel setting.

9 Stage 7 — fitting the methods

`scripts/hpc/run_benchmark_job.R` is the array worker. Task t maps to

$$\text{row} = \left\lfloor \frac{t-1}{\text{tasks per row}} \right\rfloor + 1, \qquad \text{tasks per row} = \left\lceil \frac{250}{\text{chunk}} \right\rceil,$$

with chunk = 5, so 50 tasks per row and 2,250 tasks for the full grid. Each task loads its row's cached `sim.rds`, then fits all methods to each of its five scenarios in turn, writing `results.rds` per scenario before moving on.

Eighteen methods were fitted. `susie`, `susie_inf`, `finemap`, `finemap_inf`, `abf`, `marginal_z`, `finimom`, `sparsepro`, `funmap`, `paintor`, `polyfun_oracle`, `polyfun_est`, `polyfun_ldsc`, `sbayesrc`, `beatrice`, `functional_beatrice`, `fb_pooled`, `fb_xregion`.

Four of these are reimplementations, not the upstream software. `polyfun_oracle`, `polyfun_est`, `polyfun_ldsc` and `sbayesrc` are implemented inside this package, because the published tools depend on genome-wide reference infrastructure — pre-computed UKB annotation matrices, reference LD scores, eigen-decomposed LD folders — that does not exist for simulated per-locus data.

- `polyfun_oracle` skips PolyFun's Stage 1 entirely and passes the simulator's true prior to `susie_rss()`. It is a ceiling, not PolyFun.
- `polyfun_est` is "LDSC-lite": χ^2 regressed on each variant's own annotations.
- `polyfun_ldsc` follows the canonical S-LDSC model, regressing on annotation LD scores by weighted non-negative least squares. Closest of the three.
- `sbayesrc` reimplements the SBayesRC *algorithm* (spike plus four normal slabs with annotation-modulated weights) in R.

Results for these four are evidence about the *modelling strategy*, not about the software of the same name. One difference is material enough to affect interpretation: `sbayesrc`'s PIP is the fraction of MCMC samples in which a variant occupies any non-spike component, and its smallest slab has variance 5×10^{-5} — so a variant with an effect of no consequence counts fully toward its PIP. That quantity answers "does this variant have a non-zero effect", whereas every metric here scores against "is this variant one of the S sparse causals". Its mass ratios (10–18) and top-band calibration (0.996 claimed, 0.621 realised) follow from that mismatch of definition rather than from any error, and are not comparable with the other methods'.

CARMA was dropped part-way through on measured cost: 19,000 s per scenario against 9,765 s for the other eighteen combined, i.e. 66% of total runtime, for a differentiator (LD-discrepancy outlier detection) that an all-in-sample design cannot exercise. Scenarios fitted before that change do contain CARMA results; they are retained on disk but excluded from every analysis, because partial coverage of a balanced design unbalances every stratum it touches.

The four BEATRICE variants. All share one Binary Concrete (Gumbel-Softmax) variational posterior over causal configurations and differ only in where the prior comes from:

method	prior	fitted
`beatrice`	uniform	—
`functional_beatrice`	LassoNet on annotations	per region
`fb_pooled`	shared logistic head	jointly, all regions
`fb_xregion`	shared LassoNet head	jointly, all regions

The two joint variants optimise $L(\phi, \{\psi_r\}) = \sum_r \mathrm{ELBO}_r(\psi_r; p_{0,r} = f_\phi(v_r))$ with a *single* optimiser over the shared head and all region posteriors: one backward pass per step on $\frac{1}{R}\sum_r \mathrm{loss}_r$, so every parameter moves simultaneously. It is not alternating optimisation. The equal weighting of regions follows from the $1/R$ rather than being imposed separately.

With no annotations all four reduce to the same code path, which is why they are bit-identical on both unannotated arms — a mechanical negative control, not a coincidence.

Hyperparameters for the BEATRICE family come from Optuna trial #401: `sigma_sq` = 0.0899, `sparse_concrete` = 51, `max_iter` = 2000, `n_caus` = 10, and for the functional variants `prior_regularisation` = 0.1255, `lambda_l1` = 0.1223, `hierarchy_M` = 10.15. All competitors run at their documented defaults. A separate analysis found the family's sensitivity to these values to be small (±0.004 AP across a deliberately wide hand-tuned sweep) relative to the architectural contrasts.

Per-task preflight. Before any fitting, each task imports every module the tool sources actually require — the list derived by `ast` walk, not written by hand — and aborts if any fails. All 1,800 tasks of the final run reported `preflight ok`.

10 Stage 8 — collection

`scripts/analysis/iter004_collect.R`, one array task per design row, re-derives every metric from the raw PIPs in `results.rds` and the truth in `sim.rds`. It does not read the `evaluation.rds` written during fitting.

Two details matter for correctness:

- **The scenario index comes from the directory name**, not from `fit$scenario_id`. The worker subsets `sim` to one scenario and resets that field to 1, because `evaluate_methods()` uses it as a list index; the true index survives only in the `scenario_<n>` path. Indexing by the stored field would silently attach every fit to scenario 1.
- **A fit with any non-finite PIP is marked failed, not repaired.** Its total mass and the ranking of the affected variants are undefined, so no metric derived from it is trustworthy. Imputing to zero would invent posterior mass and flatter precisely the method that failed. A separate `bad_pip` column, scoped to fits that did *not* declare an error, makes silent failure countable and distinguishable from an honest one.

Output: the L1 table, one row per (scenario, region, method), 2,031,230 rows, carrying ~ 45 metric columns.

11 Stage 9 — aggregation and analysis

`scripts/analysis/iter004_report.R` binds the L1 partials, then:

1. Joins the grid, and **recodes `NA`-as-level before any grouping**. `p_causal` is `NA` on every sparse row and `enrichment_fold` is `NA` on every unannotated row; `aggregate()`, `split()` and `table()` all drop `NA` groups silently, which would delete those arms entirely.
2. Builds L2 (replicate level: cell × iteration × method) and L3 (cell means), using a hand-rolled grouped reduction. `aggregate()` did not complete in 120 s at this scale; the replacement runs in 2.7 s and was verified to give identical results.
3. Runs the validity checks of §12.
4. Drives the three sense scripts.

The three senses are defined and derived in full in the companion analysis document; in outline, Sense V is a noise-corrected variance decomposition on cell means, Sense D a cross-fitted test of whether a cell's winner is real, and Sense G a set of paired within-replicate contrasts.

12 Validity checks performed

12.1 Automatic, on every run

1. In-sample LD only — zero rows with a reference-panel size.
2. 45 design rows present, matching the grid.
3. Balanced cells per analysed method — 5,625 for all 18.
4. Exclusions (CARMA) reported explicitly rather than silently applied.
5. No method entirely absent outside the unannotated arms.
6. `polyfun_est` = `susie` on the unannotated arms.
7. `polyfun_ldsc` = `susie` on the unannotated arms.

Checks 6 and 7 are the strongest structural tests available: both methods reduce to SuSiE analytically without annotations, so a discrepancy would indicate a fault in the annotation plumbing rather than in a method.

12.2 Additional checks performed for this report

8. Truth consistency: `n_causal` $= S$ for all 2,031,230 fit rows. Zero mismatches.
9. Grid completeness: exactly 11,250 distinct $(\text{row}, S, \phi, \text{iter})$ tuples; `region_idx` $\in \{1, 2\}$ only.
10. Metric internal consistency, per fit: $\text{tp}_t \le n_t$ and $\text{tp}_t \le S$ at $t \in \{0.5, 0.9, 0.99\}$; n_t monotone decreasing in t; AP $\in [0, 1]$; $|CS_{95}| \le p$; PIP-band counts summing to p; causal band counts summing to S. Zero violations of any.
11. Negative controls, exactly: all four BEATRICE variants identical to machine precision on both unannotated arms; `polyfun_{est,ldsc,oracle}` identical to `susie` there.

12. Directional sanity: AP strictly decreasing in S; strictly increasing in p_{causal}; sparse arm easier than sparse+infinitesimal. AP is *not* monotone in ϕ — investigated and found to be a genuine method-specific effect, not a fault.

13. Simulation code review: PVE calibration exact by construction in both architectures; oracle prior uses the identical formula to causal selection.

14. Headroom validity: the oracle's ranking advantage over the baseline is intact in all four annotated strata, so κ is defined there.

13 Incidents during execution

Recorded because a process record that omits its failures cannot be audited, and because each fix is now a guard that a future run inherits.

1. Stale grid silently reused. `params_grid.csv` is untracked and persists between iterations; a submission reused Iteration 003's 135-row grid. *Fix:* the submitter regenerates it unconditionally and prints the design before submitting.

2. CARMA dominating runtime. 66% of total compute for an untestable capability. *Fix:* removed from the method set; prior partial results retained but excluded from analysis.

3. Toolchain on a purging filesystem. `~/tools` was a symlink onto ephemeral storage subject to a rolling 30-day purge. It decayed *during* a run: activating the virtualenv kept succeeding while imports inside it stopped resolving, so seven Python-backed methods recorded 38–50% failed fits for hours without the job stopping. *Fix:* toolchain moved to permanent project storage; a per-task preflight now aborts immediately if any required import fails. Affected results were discarded and the arm refitted.

4. Dependency list written from memory. The rebuilt environment installed five packages; BEATRICE also requires `imageio`, `seaborn` and `tqdm`, and FINEMAP-inf requires `bgzip`. All fits died on `ModuleNotFoundError` after a check that had passed. *Fix:* the required imports are now derived from the tool sources by `ast` walk and each is actually imported, not merely located — an installed-but-broken package passes `find_spec` and fails at runtime.

5. `setuptools` 81 removed `pkg_resources`. An unpinned upgrade broke FINEMAP-inf, which imports it. *Fix:* pinned `setuptools<81`. Separately, `fine-mapping-inf` must be built rather than cloned (it declares a C extension) and its build needs `--no-build-isolation` because `setup.py` imports numpy at build time.

6. Silent all-NA output. `finemap_inf` returns an all-NA posterior without raising an error at $\phi = 0.6$, $S = 1$ in the sparse arm. The old evaluator crashed on it; the collect stage treated it as a successful fit. *Fix:* non-finite PIPs mark the fit failed and set a `bad_pip` flag, scoped to fits that declared no error, so silent failure is countable.

7. Scheduler instability. The final tail of the array was system-held twice for repeated launch failures, during a period when the site had disabled `qstat` for slowness. *Fix (operational):* the held subjobs were deleted and resubmitted as fresh jobs; all completed. No data was affected — completed scenarios were already on disk.

14 Reproducing the run

From the project root on the cluster:

```
# 0. toolchain (once)
DEST=/rds/general/project/neurogenomics-lab/live/tools \
  bash scripts/hpc/rebuild_toolchain.sh
bash scripts/hpc/check_toolchain.sh          # must print RESULT: PASS

# 1-7. simulate and fit  (2250 tasks, ~72h with wide concurrency)
module load R/4.5.2-gfbf-2025b
FMB_SCENARIOS_PER_TASK=5 PBS_QUEUE=v1_small72a PBS_WALLTIME=72:00:00 \
  FMB_SCRATCH=$EPHEMERAL/fmbench_iter004 \
  bash scripts/hpc/submit_benchmark_pbs.sh

# 8-9. verify, then collect and analyse  (submits itself only if the run is sound)
qsub -v PROJECT=$HOME/fine-mapping-benchmark \
  scripts/analysis/verify_and_analyse.sh
```

The verify step checks `results.rds` completeness against 11,250, scans every task log for pre-flight failures, and gates on per-method declared-failure and silent-NA rates before submitting the analysis. It refuses to proceed on a short or damaged grid.

15 Output manifest

file	level	contents
`combined_fit_metrics.rds`	L1	one row per (scenario, region, method)
`combined_replicate_metrics.rds`	L2	one row per (cell, iteration, method)
`combined_scenario_metrics.rds`	L3	cell means; the ANOVA input
`validity_checks.txt`	—	the checks of §12
`iter004_sense_v/`	—	variance decomposition
`iter004_sense_d/`	—	decision analysis, per-cell winners
`iter004_sense_g/`	—	headroom, κ, paired contrasts

L1 is the pivot: every statistic in the analysis document is re-derivable from it without refitting. The raw PIPs it was built from live under `fmbench_iter004/results/benchmark` and should be archived off ephemeral storage if any metric not already in L1 might be wanted later.